台灣預售屋總價預測：最終模型說明與程式碼

Oct 18, 2026 · @Yung-Ju Chang

1. 問題與資料特性

最終模型（對應提交檔 submission-v8-w2）的公開 MAE 為 1,705,311 元，比基準版本的 2,224,375 元降低約 23%。任務是預測台灣預售屋交易的總價，評分方式為總價的 MAE（平均絕對誤差）。

分析資料時發現三個關鍵特性，整個方法都圍繞它們設計：

1. **test 在時間上位於 train 之後。** train 的交易日期到民國 113 年 7 月，test 為 113 年 8 月到 114 年 7 月，模型要預測的是「未來 1～12 個月」的價格。
2. **test 有 63% 的交易屬於 train 從未出現的建案。** 模型不能只靠記住建案價格，必須從地段、樓層、坪數等資訊推估。
3. **test 有 1,408 筆「桃園航空城安置住宅」，train 一筆都沒有。** 這是政府配售給徵收戶的住宅，價格遠低於市價。

2. 預測目標

模型不直接預測總價，而是預測「每平方公尺單價取對數」，預測後再乘回面積還原成總價。

```
y = \log\left(\frac{\text{總價}}{\text{有效面積}}\right)
```

有效面積優先用建物移轉面積；若為 0，依序改用土地面積、車位面積。這樣做的理由：

- 總價從數百萬到上億都有，跨度太大；換成單價再取對數後分布穩定許多。
- 訓練時去掉單價最極端的上下各 0.2%，避免少數異常交易干擾模型。

3. 核心設計：模擬「預測未來」的歷史價格特徵

訓練資料的每一筆，都隨機假裝「資料只更新到 1～12 個月以前」，只用那個時間點以前的成交計算歷史價格。這讓模型在訓練時面對的情況，與 test 一致：資訊有延遲，而且常查不到建案本身的紀錄。預測 test 時則使用 train 的全部歷史。

若不這樣做，訓練時每筆都看得到上個月的最新成交，模型會過度依賴這些資訊。早期版本即因此在隨機切分驗證得到 MAE 64 萬元，test 卻高達 276 萬元。

歷史價格採用平滑平均，筆數少時偏向上一層的值，不會被一兩筆極端成交帶偏：

```
\text{平滑平均} = \frac{\text{該 key 歷史總和} + m \times \text{上一層的值}}
{\text{筆數} + m}
```

沒看過的建案會分層退回：建案 → 路段 → 路 → 行政區 → 全國。統計以「每個 key 每月的累積總和與筆數」加上 merge_asof 查表完成，完全向量化，46 萬筆資料約十幾秒即可算完。

4. 兩套特徵

模型使用兩套差異很大的特徵，讓模型「錯在不同地方」，混合後互相抵消。兩套都只用非價格欄位建立基本特徵，價格資訊只透過第 3 節的時間式歷史價格進入模型。

類別	特徵 A（舊版特徵）	特徵 B（v5 特徵）
地址解析	正規表達式切出縣市、行政區、路名、路段	另統一全半形與「臺/台」、去除重複縣市區、處理里鄰、抓路口第二條路、地號地址改抓地段；所有 key 加上縣市與區前綴
時間	絕對時間 t、交易月份	交易月份、資料延遲月數、距上次成交月數
建物	樓層解析（中文數字）、樓層比例、是否頂樓、面積與面積比例、格局	同左
備註旗標	裝潢、家具、親友交易、車位、店面等	另加地上權、毛胚屋、合建、露台、雨遮、差額減收等
建案規模	戶數、最高樓層、平均面積、開賣多久	同左
歷史價格	區、路、路段、建案、棟、區×建物型態、區×樓高；含近 18 個月版本	全國、市、區、地段、路、路段、建案、棟；全期、近 12 月、近 6 月；近期漲跌趨勢
新建案行情	無	只統計「開賣 6 個月內」的成交，算出同市、同區、同路的新案價格水準

建案規模特徵使用 train 與 test 的非價格欄位計算，不涉及答案，沒有洩漏問題。

5. 七個模型

最終使用 7 個梯度提升樹回歸模型，特徵 A 訓練 4 個、特徵 B 訓練 3 個。

名稱	特徵	演算法	目標函數	說明
old_hgb	A	sklearn HistGradientBoosting	絕對誤差	early stopping，最多 3000 棵樹
old_lgb	A	LightGBM	L1	隨機延遲種子 42

名稱	特徵	演算法	目標函數	說明
old_lgb_s7	A	LightGBM	L1	隨機延遲種子 7，產生另一組歷史特徵
old_l2	A	LightGBM	L2	隨機延遲種子 42
v5_l1_w	B	LightGBM	L1	依總價加權
v5_l1_nok	B	LightGBM	L1	行政區不當類別特徵
v5_l2	B	LightGBM	L2	

- **以 L1 為主：** L1 預測的是中位數，最符合 MAE 評分。
- **建案與路名不直接當類別特徵：** 早期版本直接使用時，模型會「背」建案價格，在 test 上嚴重失準；最終版只透過歷史價格使用它們。

6. 混合方式

7 個模型的預測取加權幾何平均：對 log 單價取加權平均後轉回總價。特徵 A 與特徵 B 兩側的總權重各為 0.5。

模型	權重
old_hgb	0.13
old_lgb	0.17
old_lgb_s7	0.17
old_l2	0.05
v5_l1_w	0.10
v5_l1_nok	0.19
v5_l2	0.19

權重依多次上傳結果逐步調整：拿掉 old_lgb 使 MAE 變差約 2.2 萬元；加重 v5_l1_w 變差約 2.7 萬元；加重舊版 LightGBM 家族並降低 v5_l1_w，連續兩次都有進步。

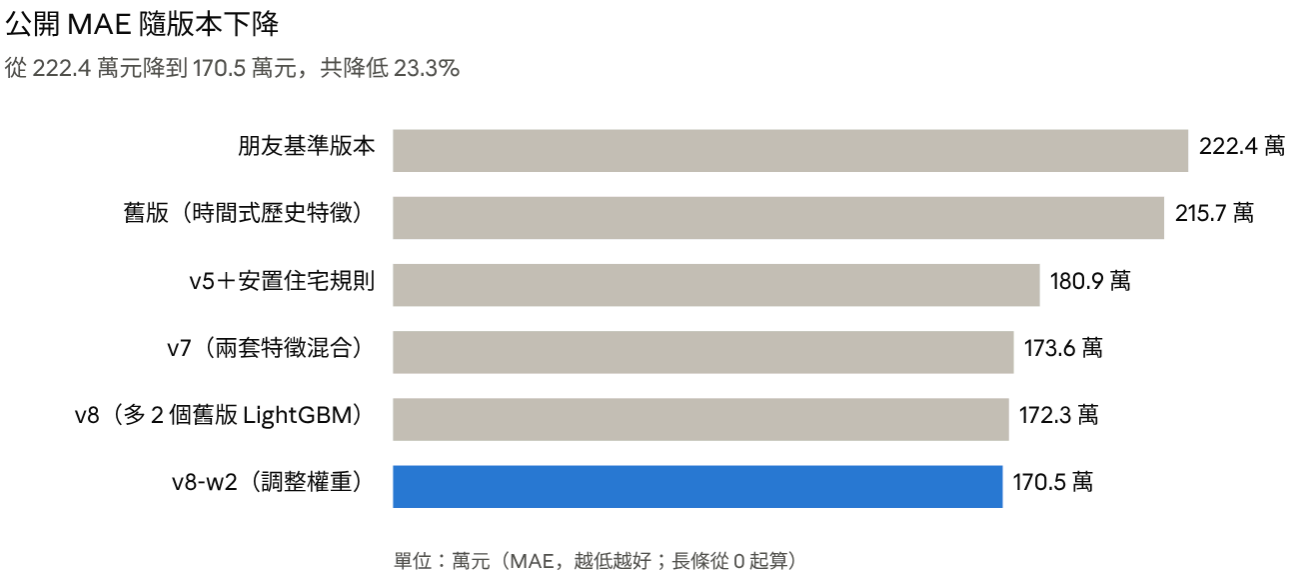
7. 安置住宅規則

備註含「安置」的 1,408 筆交易不使用模型預測，改用規則估價；加入此規則後 MAE 降低約 33 萬元。train 中沒有安置住宅，模型只能把它們當一般新成屋估價，會大幅高估。

```
\text{總價} = \frac{\text{建物面積} - \text{車位面積}}{3.305785} \times
180{,}000 + \text{車位數} \times 1{,}000{,}000 - \text{差額減收金額}
```

- **每坪單價：** 參考公開新聞報導與實價登錄網站的整體配售價格水準（每坪約 18～22 萬元），再以上傳結果確認 18 萬元最佳。
- **差額減收：** 部分交易備註記載「原房屋價款 X 元，差額減收後為 Y 元」，代表實際付款少了 X − Y 元，因此從總價扣除。
- **揭露：** 此單價參考公開的整體行情並以排行榜微調，屬於領域知識，並非查詢個別交易的成交價。

8. 成績演進



Kaggle 公開排行榜分數 · 依上傳結果整理

降幅最大的一步是加入安置住宅規則；其後的進步來自兩套特徵混合與增加模型差異性。

9. 主要發現

1. **驗證方式必須與 test 情況一致。** 隨機切分驗證得到 MAE 64 萬元，test 卻為 276 萬元；改用時間切分並讓歷史價格隨機延遲後，模型才學得到 test 真正面對的情況。
2. **找出 train 沒有、test 才有的資料類型，效益最大。** 安置住宅一條規則就讓 MAE 降低約 33 萬元，超過任何模型調整。
3. **模型之間的差異性很重要，但不是越多越好。** 兩套特徵的模型單獨都在 180 萬元左右，混合後降到 173 萬元；但加入不夠好的模型會退步，例如一次嘗試從 181 萬元變成 187 萬元。
4. **只有一兩個參數要調時，可用排行榜確認，但要控制次數。** 安置住宅單價與混合權重都只用少量上傳即找到較佳值。

10. 執行環境與重現方式

將第 11 節程式存成 final_model.py，與資料檔放在同一資料夾，執行 python final_model.py，約 20 分鐘後產生 submission.csv。

- **資料檔：** train-v2.csv、valid-v2.xlsx、test-reindex-test-v2.2.xlsx。
- **套件：** pandas、numpy、lightgbm、scikit-learn、openpyxl（讀取 xlsx）。
- **從頭訓練：** 程式訓練全部 7 個模型，不讀取任何先前的預測檔，可在乾淨環境完整重現。
- **數值差異：** LightGBM 多執行緒運算會有極小差異，重跑分數可能與 1,705,311 元相差數十到數百元。
- **參數來源：** 混合權重與安置住宅單價依排行榜結果調整，並非純粹從訓練資料學得。

11. 完整程式碼（final_model.py）

以下為單一檔案的完整程式，分兩段列印；兩段依序接在一起即為完整的 final_model.py。

第 1 段：設定、共用工具、特徵 A

```
"""
台灣預售屋總價預測（評分：總價 MAE）— 最終版本（對應 submission-v8-w2）

流程：
1. 讀取 train / valid / test，合併 train+valid 當訓練資料
2. 建立兩套特徵（A：舊版特徵；B：v5 特徵），兩套都含「時間式歷史價格」
   - 訓練資料每列隨機假設資料只更新到 1~12 個月前，模擬 test（未來 1~12 個月、多為新建案）
3. 訓練 7 個模型（目標 = log(總價 / 有效面積)）
4. 加權幾何平均混合
5. 安置住宅（train 中不存在）改用規則估價
6. 輸出 submission.csv

需要的檔案（與本程式同資料夾）：
  train-v2.csv、valid-v2.xlsx、test-reindex-test-v2.2.xlsx
執行：python final_model.py（約 20 分鐘）
"""
import re
import time
import unicodedata
from pathlib import Path

import lightgbm as lgb
import numpy as np
import pandas as pd
from sklearn.ensemble import HistGradientBoostingRegressor
```

```
# ============================================================
# 0. 設定
# ============================================================
ROOT = Path(__file__).resolve().parent
TRAIN_FILE = ROOT / "train-v2.csv"
VALID_FILE = ROOT / "valid-v2.xlsx"
TEST_FILE = ROOT / "test-reindex-test-v2.2.xlsx"
OUT_FILE = ROOT / "submission.csv"

LAG_MAX = 12              # 歷史特徵隨機延遲的最大月數
PER_PING = 180_000        # 安置住宅每坪單價（不含車位）
PARK = 1_000_000          # 安置住宅每個車位價格
PING = 3.305785           # 1 坪 = 3.305785 平方公尺

# 最終混合權重（加權幾何平均）
WEIGHTS = {
    "old_hgb": 0.13,
    "old_lgb": 0.17,
    "old_lgb_s7": 0.17,
    "old_l2": 0.05,
    "v5_l1_w": 0.10,
    "v5_l1_nok": 0.19,
    "v5_l2": 0.19,
}

OLD_LGB_PARAMS = dict(
    objective="regression_l1", learning_rate=0.1, num_leaves=127,
min_child_samples=40,
    reg_lambda=1.0, max_bin=255, metric="None", n_jobs=-1, verbosity=-1,
)
OLD_LGB_ITER = 2000
OLD_HGB_MAX_ITER = 3000

V5_PARAMS = dict(
    learning_rate=0.1, num_leaves=127, min_child_samples=40, subsample=0.8,
    subsample_freq=1, colsample_bytree=0.7, reg_lambda=1.0, max_bin=255,
    metric="None", n_jobs=-1, verbosity=-1,
)
V5_VARIANTS = [
    dict(name="v5_l1_w", objective="regression_l1", weight=True,
kdist_cat=True, n_iter=2400),
    dict(name="v5_l1_nok", objective="regression_l1", weight=False,
kdist_cat=False, n_iter=2400),
    dict(name="v5_l2", objective="regression", weight=False, kdist_cat=True,
n_iter=1500),
]
```

```
# ============================================================
# 1. 共用工具
# ============================================================
def read_any(p):
    """讀 csv / xlsx，並去除欄名中的空白。"""
    if p.suffix == ".csv":
        df = pd.read_csv(p, encoding="utf-8-sig", low_memory=False)
    else:
        df = pd.read_excel(p)
    df.columns = [str(c).replace(" ", "").replace("\u3000", "") for c in
df.columns]
    return df


def load():
    """合併 train+valid 為訓練資料；去除混入的英文標題列、重複編號、非正價格。"""
    t0 = time.time()
    tr, va, te = read_any(TRAIN_FILE), read_any(VALID_FILE),
read_any(TEST_FILE)
    full = pd.concat([tr, va], ignore_index=True)
    full = full[full["編號"].astype(str) != "serial number"]
    full = full.drop_duplicates("編號")
    full["總價元"] = pd.to_numeric(full["總價元"], errors="coerce")
    full = full[full["總價元"] > 0].reset_index(drop=True)
    print(f"讀檔: 訓練={len(full):,} 筆, 測試={len(te):,} 筆 ({time.time() -
t0:.0f}s)")
    return full, te.reset_index(drop=True)


CN = {"一": 1, "二": 2, "三": 3, "四": 4, "五": 5, "六": 6, "七": 7, "八": 8,
"九": 9}
FLOOR_RE = re.compile(r"(地下)?([一二三四五六七八九十]+)層")


def cn2int(s):
    """中文數字轉整數（支援到九十九）。"""
    if "十" in s:
        a, b = s.split("十", 1)
        return (CN.get(a, 1) if a else 1) * 10 + (CN.get(b, 0) if b else 0)
    return CN.get(s, np.nan)


def parse_floor_cn(s):
    """移轉層次 → (第一個樓層, 最高樓層, 移轉幾層)。"""
```

```
    if not isinstance(s, str):
        return np.nan, np.nan, np.nan
    fl = [(-1 if m.group(1) else 1) * cn2int(m.group(2)) for m in
FLOOR_RE.finditer(s)]
    return (fl[0], max(fl), len(fl)) if fl else (np.nan, np.nan, 0)


def num(df, c):
    return pd.to_numeric(df[c], errors="coerce") if c in df else
pd.Series(np.nan, index=df.index)


def area_eff(df):
    """有效面積：建物面積 > 土地面積 > 車位面積 > 1，用來把總價換成單價。"""
    a_b = num(df, "建物移轉總面積平方公尺").fillna(0)
    a_l = num(df, "土地移轉總面積平方公尺").fillna(0)
    a_p = num(df, "車位移轉總面積平方公尺").fillna(0)
    return np.where(a_b > 0, a_b, np.where(a_l > 0, a_l, np.where(a_p > 0,
a_p, 1.0)))


def _cum_stats(k, mi, y):
    """每個 key 依月份累積的 (總和, 筆數)。"""
    g = pd.DataFrame({"k": np.asarray(k, dtype=object), "mi": np.asarray(mi,
float),
                      "y": np.asarray(y, float)}).dropna()
    st = g.groupby(["k", "mi"])["y"].agg(["sum",
"count"]).reset_index().sort_values(["k", "mi"])
    st["csum"] = st.groupby("k")["sum"].cumsum()
    st["ccnt"] = st.groupby("k")["count"].cumsum()
    return st[["k", "mi", "csum",
"ccnt"]].sort_values("mi").reset_index(drop=True)


def _lookup(st, k, c):
    """查詢 key k 在月份 <= c 的累積 (總和, 筆數, 最後成交月份)。"""
    q = pd.DataFrame({"k": np.asarray(k, dtype=object), "c": np.asarray(c,
float),
                      "i": np.arange(len(k))})
    q = q[q["k"].notna() & q["c"].notna()].sort_values("c")
    s, n, last = np.zeros(len(k)), np.zeros(len(k)), np.full(len(k), np.nan)
    if len(q) and len(st):
        r = st.assign(lastmi=st["mi"]).rename(columns={"mi": "c"})
        mg = pd.merge_asof(q, r, on="c", by="k", direction="backward")
        idx = mg["i"].values
        s[idx] = mg["csum"].fillna(0).values
```

```
        n[idx] = mg["ccnt"].fillna(0).values
        last[idx] = mg["lastmi"].values
    return s, n, last


def target(f):
    """目標 = log(總價 / 有效面積)；去除最極端的 0.2% 單價。"""
    y = np.log(f["總價元"] / f["area_eff"])
    lo, hi = y.quantile([0.002, 0.998])
    keep = y.between(lo, hi).to_numpy()
    return f[keep].reset_index(drop=True), y[keep].reset_index(drop=True)


# ============================================================
# 2. 特徵 A（舊版特徵）
# ============================================================
O_CAT = ["city", "交易標的", "建物型態", "主要用途", "主要建材", "車位類別",
         "都市土地使用分區", "非都市土地使用分區", "非都市土地使用編定", "建物現況格局-
隔間"]
O_KEYS = ["k_dist", "k_road", "k_seg", "k_proj", "k_bld"]
O_ROAD_RE = re.compile(r"^(.+?(?:路|街|大道))([一二三四五六七八九十]+段)?")
O_REMARK = {
    "rm_deco": "裝潢", "rm_furn": "傢俱|家具|家電", "rm_rel": "親友|特殊關係|員工|
關係人",
    "rm_urgent": "急", "rm_rooftop": "頂樓|露台|平台", "rm_park": "車位",
"rm_shop": "店面|商業",
}


def old_base(df):
    """不使用房價的基本特徵（每列只看自己）。"""
    f = pd.DataFrame(index=df.index)
    addr = df["土地位置建物門牌"].astype(str).str.replace("台", "臺")
    city = addr.str[:3]
    f["city"] = city
    dist = df["鄉鎮市區"].fillna(addr.str[3:6]).astype(str)
    f["k_dist"] = city + dist
    rest = pd.Series([a.split(d, 1)[-1] for a, d in zip(addr, dist)],
index=df.index)
    rs = rest.str.extract(O_ROAD_RE)
    road, seg = rs[0], rs[1].fillna("")
    f["k_road"] = np.where(road.notna(), f["k_dist"] + "|" + road.fillna(""),
None)
    f["k_seg"] = np.where(road.notna(), f["k_road"].astype(str) + seg, None)
    f["is_cross"] = rest.str.contains("交叉|與").astype(int)
    f["is_lot"] = rest.str.contains("地號").astype(int)
```

```
    proj = df["建案名稱"].fillna("").astype(str).str.strip()
    f["k_proj"] = np.where(proj != "", f["k_dist"] + "|" + proj, None)
    bld = df["棟及號"].fillna("").astype(str).str.split("-
").str[0].str.strip()
    f["k_bld"] = np.where((proj != "") & (bld != ""), f["k_proj"].astype(str)
+ "|" + bld, None)

    d = num(df, "交易年月日")
    d = d.where((d > 1000000) & (d < 1200000))
    y, m = d // 10000, (d // 100) % 100
    f["t"] = y + (m - 1) / 12
    f["month"] = m

    nums = df["交易筆棟數"].astype(str).str.extract(r"土地(\d+)建物(\d+)車位
(\d+)").astype(float)
    f["n_land"], f["n_bld"], f["n_park"] = nums[0], nums[1], nums[2]
    fl = df["移轉層次"].map(parse_floor_cn)
    f["floor"] = [a for a, _, _ in fl]
    f["floor_max"] = [b for _, b, _ in fl]
    f["n_floor"] = [c for _, _, c in fl]
    f["floor_all"] = df["移轉層次"].astype(str).str.contains("全").astype(int)
    f["total_floor"] = num(df, "總樓層數")
    f["floor_ratio"] = f["floor"] / f["total_floor"].replace(0, np.nan)
    f["is_top"] = (f["floor_max"] == f["total_floor"]).astype(int)

    a_b = num(df, "建物移轉總面積平方公尺").fillna(0)
    a_l = num(df, "土地移轉總面積平方公尺").fillna(0)
    a_p = num(df, "車位移轉總面積平方公尺").fillna(0)
    f["a_bld"], f["a_land"], f["a_park"] = a_b, a_l, a_p
    f["a_main"] = (a_b - a_p).clip(lower=0)
    f["log_a_bld"] = np.log1p(a_b)
    f["land_ratio"] = a_l / a_b.replace(0, np.nan)
    f["park_ratio"] = a_p / a_b.replace(0, np.nan)
    for c in ["房", "廳", "衛"]:
        f[c] = num(df, f"建物現況格局-{c}")
    f["rooms"] = f[["房", "廳", "衛"]].sum(axis=1)
    f["a_per_room"] = f["a_main"] / f["房"].replace(0, np.nan)

    rm = df["備註"].fillna("").astype(str)
    for k, pat in O_REMARK.items():
        f[k] = rm.str.contains(pat).astype(int)
    f["has_remark"] = (rm != "").astype(int)
    f["cancel"] = df["解約情形"].notna().astype(int) if "解約情形" in df else 0
    for c in O_CAT[1:]:
        f[c] = df[c].astype(str) if c in df else "nan"
    f["area_eff"] = area_eff(df)
```

```
    return f


def old_project(f_all):
    """建案規模特徵（用 train+test 的非價格欄位，不會洩漏）。"""
    g = f_all.dropna(subset=["k_proj"]).groupby("k_proj")
    agg = pd.DataFrame({
        "proj_units": g.size(),
        "proj_max_floor": g["total_floor"].max(),
        "proj_mean_area": g["a_bld"].mean(),
        "proj_med_rooms": g["房"].median(),
        "proj_first_t": g["t"].min(),
        "proj_park_rate": g["n_park"].apply(lambda s: (s > 0).mean()),
    })
    out = f_all[["k_proj"]].join(agg, on="k_proj").drop(columns="k_proj")
    out["t_since_launch"] = f_all["t"] - out["proj_first_t"]
    out["area_vs_proj"] = f_all["a_bld"] / out["proj_mean_area"]
    out["dist_units"] = f_all.groupby("k_dist")["k_dist"].transform("size")
    return out


def old_prepare(full, test):
    t0 = time.time()
    ff, ft = old_base(full), old_base(test)
    n = len(ff)
    fa = pd.concat([ff, ft], ignore_index=True)
    fa = pd.concat([fa, old_project(fa)], axis=1)
    ff, ft = fa.iloc[:n].reset_index(drop=True),
fa.iloc[n:].reset_index(drop=True)
    ff["總價元"] = full["總價元"].astype(float).values
    print(f"[特徵 A] 基本特徵完成 ({time.time() - t0:.0f}s)")
    return ff, ft


O_RECENT = 18
# (輸出名稱, key, 平滑強度, 上一層, 時間窗月數, 是否輸出筆數)
O_TE_SPEC = [
    ("k_dist", "k_dist", 20, None, None, True),
    ("k_road", "k_road", 20, "k_dist", None, True),
    ("k_seg", "k_seg", 20, "k_road", None, True),
    ("k_proj", "k_proj", 5, "k_seg", None, True),
    ("k_bld", "k_bld", 5, "k_proj", None, True),
    ("dist_recent", "k_dist", 20, "k_dist", O_RECENT, True),
    ("road_recent", "k_road", 20, "dist_recent", O_RECENT, False),
    ("seg_recent", "k_seg", 20, "road_recent", O_RECENT, False),
    ("dist_type", "dist_type", 20, "k_dist", None, False),
```

```
    ("road_type", "road_type", 20, "dist_type", None, False),
    ("dist_fl", "dist_fl", 20, "k_dist", None, False),
    ("dist_fl_recent", "dist_fl", 20, "dist_fl", O_RECENT, False),
]


def old_keys(f):
    fb = pd.cut(f["total_floor"], [-1, 5, 10, 15, 20, 30, 999],
labels=False).astype(str)
    return {
        "k_dist": f["k_dist"], "k_road": f["k_road"], "k_seg": f["k_seg"],
        "k_proj": f["k_proj"], "k_bld": f["k_bld"],
        "dist_type": f["k_dist"] + "|" + f["建物型態"],
        "road_type": f["k_road"] + "|" + f["建物型態"],
        "dist_fl": f["k_dist"] + "|" + fb,
    }


def old_te_block(f_fit, y, f_q, cutoff):
    """對 f_q 每列，用 f_fit 中月份 <= cutoff 的成交計算平滑平均（沒資料則退回上一
層）。"""
    mi_fit = np.round(f_fit["t"] * 12)
    kf, kq = old_keys(f_fit), old_keys(f_q)
    stats = {k: _cum_stats(v, mi_fit, y) for k, v in kf.items()}
    gmean = float(y.mean())
    out = pd.DataFrame(index=f_q.index)
    for name, key, m, parent, win, with_cnt in O_TE_SPEC:
        s, n, _ = _lookup(stats[key], kq[key], cutoff)
        if win:
            s0, n0, _ = _lookup(stats[key], kq[key], cutoff - win)
            s, n = s - s0, n - n0
        prior = out[f"te_{parent}"].values if parent else gmean
        with np.errstate(invalid="ignore", divide="ignore"):
            out[f"te_{name}"] = np.where(n > 0, (s + m * prior) / (n + m),
prior)
        if with_cnt:
            out[f"cnt_{name}"] = n
    out["dist_trend"] = out["te_dist_recent"] - out["te_k_dist"]
    return out


def old_build(of, ot, seed):
    """產生特徵 A 的模型矩陣。訓練列的歷史特徵隨機延遲 1~12 個月；測試列用全部歷史。"""
    t0 = time.time()
    of, y = target(of)
    rng = np.random.default_rng(seed)
```

```
    mi = np.round(of["t"] * 12).to_numpy()
    lag = rng.integers(1, LAG_MAX + 1, size=len(of))
    enc_fit = old_te_block(of, y, of, mi - lag)
    enc_app = old_te_block(of, y, ot, np.full(len(ot), np.nanmax(mi)))
    cats = {c: list(of[c].astype(str).value_counts().index[:250]) for c in
O_CAT}
    drop = set(O_KEYS + ["area_eff", "總價元"])

    def mat(f, enc):
        X = pd.concat([f.drop(columns=[c for c in drop if c in
f.columns]).reset_index(drop=True),
                       enc.reset_index(drop=True)], axis=1)
        for c in O_CAT:
            X[c] = pd.Categorical(X[c].astype(str), categories=cats[c])
        return X

    X_fit, X_app = mat(of, enc_fit), mat(ot, enc_app)
    print(f"[特徵 A] 歷史特徵(seed={seed}) 完成 ({time.time() - t0:.0f}s), 特徵
數 {X_fit.shape[1]}")
    return X_fit, y, X_app
```

第 2 段：特徵 B、模型、安置住宅規則、主程式

```
# ============================================================
# 3. 特徵 B（v5 特徵）
# ============================================================
CAT_COLS = ["k_city", "k_dist", "交易標的", "建物型態", "主要用途", "主要建材",
"車位類別",
            "都市土地使用分區", "非都市土地使用分區", "非都市土地使用編定", "建物現況格
局-隔間"]
CITIES = ["台北市", "新北市", "桃園市", "台中市", "台南市", "高雄市", "新竹市", "嘉
義市",
          "基隆市", "新竹縣", "苗栗縣", "彰化縣", "南投縣", "雲林縣", "嘉義縣", "屏
東縣",
          "宜蘭縣", "花蓮縣", "台東縣", "澎湖縣", "金門縣", "連江縣"]
OLD_CITY = {"桃園縣": "桃園市", "台北縣": "新北市", "台中縣": "台中市",
            "台南縣": "台南市", "高雄縣": "高雄市"}
CITY_RE = re.compile("^(" + "|".join(CITIES + list(OLD_CITY)) + ")")
ADMIN_RE = re.compile(r"^[\u4e00-\u9fff]{1,3}?[區鄉鎮市]")
VILLAGE_RE = re.compile(
    r"^(?:[\u4e00-\u9fff]{1,3}[里村]\d+鄰|[\u4e00-\u9fff]{2}里(?=[\u4e00-
\u9fff]{1,6}(?:路|街|大道)))")
ROAD_RE = re.compile(r"^([\u4e00-\u9fff]{1,8}?(?:大道|路|街))([一二三四五六七八九
十]+段)?")
ROAD2_RE = re.compile(r"(?:與|及|和|跟|、|/|&|vs|VS|x|X)([\u4e00-\u9fff]{1,8}?
(?:大道|路|街))")
```

```
SEC_RE = re.compile(r"([\u4e00-\u9fff]{2,6}?段)")
CN_DIGIT = "一二三四五六七八九"
REMARK_KW = {
    "rm_resettle": "安置", "rm_discount": "減收", "rm_superficies": "地上權",
    "rm_raw": "毛胚", "rm_joint": "合建", "rm_terrace": "露台|露臺", "rm_deco":
"裝潢",
    "rm_furn": "傢俱|家具|家電", "rm_rel": "親友|特殊關係|員工|關係人", "rm_rain":
"雨遮",
    "rm_change": "更正|撤銷|重新申報", "rm_park": "車位", "rm_shop": "店面|店鋪|商
業",
}


def norm_text(s):
    """地址正規化：全半形、臺→台、去空白、路段數字轉中文。"""
    s = unicodedata.normalize("NFKC", str(s))
    s = re.sub(r"\s+", "", s).replace("臺", "台")
    return re.sub(r"(?<!\d)([1-9])段", lambda m: CN_DIGIT[int(m.group(1)) -
1] + "段", s)


def strip_admin(rest, dist):
    """去掉開頭重複的縣市、行政區（如「新竹市新竹市東區」「龜山區龜山區」）。"""
    for _ in range(4):
        m = CITY_RE.match(rest)
        if m:
            rest = rest[m.end():]
            continue
        if dist and rest.startswith(dist):
            rest = rest[len(dist):]
            continue
        m = ADMIN_RE.match(rest)
        if m and (m.group(0)[-1] in "區鄉鎮" or len(m.group(0)) == 3):
            rest = rest[m.end():]
            continue
        break
    return rest


def parse_rest(r):
    """從縣市區之後的地址取出：路名、路段、路口第二條路、地段。"""
    r = VILLAGE_RE.sub("", r, count=1)
    r = re.sub(r"^\d+鄰", "", r)
    road = seg = road2 = sec = None
    after = r
    m = ROAD_RE.match(r)
```

```
    if m:
        road, seg = m.group(1), m.group(2) or ""
        after = r[m.end():]
        m2 = ROAD2_RE.search(after)
        if m2 and m2.group(1) != road:
            road2 = m2.group(1)
    ms = SEC_RE.search(after)
    if ms:
        s = ms.group(1)
        if not re.search(r"[路街巷弄]", s) and not re.fullmatch(r"[一二三四五六
七八九十]+段", s):
            sec = s
    return road, seg, road2, sec


def v5_base(df):
    """不使用房價的基本特徵（每列只看自己）。"""
    f = pd.DataFrame(index=df.index)
    addr = df["土地位置建物門牌"].map(norm_text)
    dist = df["鄉鎮市區"].fillna("").astype(str)
    cities, rests = [], []
    for a, d in zip(addr, dist):
        m = CITY_RE.match(a)
        cities.append(OLD_CITY.get(m.group(1), m.group(1)) if m else None)
        rests.append(strip_admin(a[m.end():] if m else a, d))
    parsed = [parse_rest(r) for r in rests]
    f["_city"], f["_dist"] = cities, dist.values
    f["_road"] = [p[0] for p in parsed]
    f["_seg"] = [p[1] for p in parsed]
    f["_road2"] = [p[2] for p in parsed]
    f["_sec"] = [p[3] for p in parsed]
    rest = pd.Series(rests, index=df.index)
    f["is_cross"] = rest.str.contains(r"交叉|交岔|交口|路口|與|
及|vs|VS|、").astype("int8")
    f["is_lot"] = rest.str.contains("地號").astype("int8")
    f["has_number"] = rest.str.contains(r"\d+號").astype("int8")
    proj = df["建案名稱"].fillna("").astype(str).str.strip()
    f["_proj"] = proj.where(~proj.isin(["", "nan", "無", "空白", "None"]))
    bn = df["棟及號"].fillna("").astype(str).str.strip()
    f["_bld"] = bn.str.extract(r"^(.+?)棟")[0].fillna(bn.str.split("-
").str[0])

    d = num(df, "交易年月日")
    d = d.where((d > 1000000) & (d < 1200000))
    f["mi"] = (d // 10000) * 12 + (d // 100) % 100
    f["month"] = (d // 100) % 100
```

```
    n = df["交易筆棟數"].astype(str).str.extract(r"土地(\d+)建物(\d+)車位
(\d+)").astype(float)
    f["n_land"], f["n_bld"], f["n_park"] = n[0], n[1], n[2]
    fl = df["移轉層次"].map(parse_floor_cn)
    f["floor"] = [a for a, _, _ in fl]
    f["floor_max"] = [b for _, b, _ in fl]
    f["n_floor"] = [c for _, _, c in fl]
    f["floor_all"] = df["移轉層
次"].astype(str).str.contains("全").astype("int8")
    f["total_floor"] = num(df, "總樓層數")
    f["floor_ratio"] = f["floor"] / f["total_floor"].replace(0, np.nan)
    f["is_top"] = (f["floor_max"] == f["total_floor"]).astype("int8")

    a_b = num(df, "建物移轉總面積平方公尺").fillna(0)
    a_l = num(df, "土地移轉總面積平方公尺").fillna(0)
    a_p = num(df, "車位移轉總面積平方公尺").fillna(0)
    f["a_bld"], f["a_land"], f["a_park"] = a_b, a_l, a_p
    f["a_main"] = (a_b - a_p).clip(lower=0)
    f["log_a_bld"] = np.log1p(a_b)
    f["land_ratio"] = a_l / a_b.replace(0, np.nan)
    f["park_ratio"] = a_p / a_b.replace(0, np.nan)
    for c in ["房", "廳", "衛"]:
        f[c] = num(df, f"建物現況格局-{c}")
    f["a_per_room"] = f["a_main"] / f["房"].replace(0, np.nan)

    rm = df["備註"].fillna("").astype(str)
    for k, pat in REMARK_KW.items():
        f[k] = rm.str.contains(pat).astype("int8")
    f["has_remark"] = (rm != "").astype("int8")
    disc = rm.str.extract(r"原房屋價款(\d+)元.*?減收後之房屋價款為(\d+)
元").astype(float)
    f["disc_ratio"] = disc[1] / disc[0]
    f["disc_amt"] = disc[0] - disc[1]
    f["cancel"] = df["解約情形"].notna().astype("int8") if "解約情形" in df
else 0
    for c in CAT_COLS[2:]:
        f[c] = df[c].astype(str) if c in df else "nan"
    f["area_eff"] = area_eff(df)
    return f


def add_keys(f):
    """建立歷史價格用的 key，全部加上「縣市+區」前綴，避免全台同名混在一起。"""
    kd = f["_city"].fillna("?") + f["_dist"]
```

```
    def kk(v):
        return np.where(v.notna(), kd + "|" + v.fillna("").astype(str), None)

    f["k_all"] = "all"
    f["k_city"] = f["_city"].fillna("?")
    f["k_dist"] = kd
    f["k_road"] = kk(f["_road"])
    f["k_seg"] = np.where(f["_road"].notna(), f["k_road"].astype(str) +
f["_seg"].fillna(""), None)
    f["k_road2"] = kk(f["_road2"])
    f["k_sec"] = kk(f["_sec"])
    f["k_proj"] = kk(f["_proj"])
    f["k_bld"] = np.where(f["k_proj"].notna() & f["_bld"].fillna("").ne(""),
                          f["k_proj"].astype(str) + "|" +
f["_bld"].fillna(""), None)
    f["k_dtype"] = kd + "|" + f["建物型態"]
    fb = pd.cut(f["total_floor"], [-1, 5, 10, 15, 20, 30, 999],
labels=False).astype(str)
    f["k_dfl"] = kd + "|" + fb
    return f


def add_project_feats(f):
    """建案規模特徵（用 train+test 的非價格欄位，不會洩漏）。"""
    g = f.dropna(subset=["k_proj"]).groupby("k_proj")
    agg = pd.DataFrame({
        "proj_units": g.size(),
        "proj_mean_area": g["a_bld"].mean(),
        "proj_max_floor": g["total_floor"].max(),
        "proj_first_mi": g["mi"].min(),
        "proj_park_rate": g["n_park"].apply(lambda s: (s > 0).mean()),
    })
    f = f.join(agg, on="k_proj")
    f["proj_age"] = f["mi"] - f["proj_first_mi"]
    f["area_vs_proj"] = f["a_bld"] / f["proj_mean_area"]
    f["dist_units"] = f.groupby("k_dist")["k_dist"].transform("size")
    return f


def add_new_keys(f):
    """只保留「建案開賣 6 個月內」的成交，作為新建案行情的統計樣本。"""
    new = f["proj_age"].le(6) & f["k_proj"].notna()
    f["k_citynew"] = np.where(new, f["k_city"], None)
    f["k_distnew"] = np.where(new, f["k_dist"], None)
    f["k_dtypenew"] = np.where(new, f["k_dtype"], None)
    f["k_roadnew"] = np.where(new & f["k_road"].notna(), f["k_road"], None)
```

```
    return f


def v5_prepare(full, test):
    t0 = time.time()
    ff, ft = v5_base(full), v5_base(test)
    ft["mi"] = ft["mi"].fillna(ft["mi"].median())
    n = len(ff)
    allf = pd.concat([ff, ft], ignore_index=True)
    # 地址沒寫縣市的，用該區最常見的縣市補上
    mp = allf.dropna(subset=["_city"]).groupby("_dist")["_city"].agg(lambda
s: s.value_counts().index[0])
    allf["_city"] = allf["_city"].fillna(allf["_dist"].map(mp))
    allf = add_new_keys(add_project_feats(add_keys(allf)))
    ff = allf.iloc[:n].reset_index(drop=True)
    ft = allf.iloc[n:].reset_index(drop=True)
    ff["總價元"] = full["總價元"].astype(float).values
    ff = ff.dropna(subset=["mi"]).reset_index(drop=True)
    print(f"[特徵 B] 基本特徵完成 ({time.time() - t0:.0f}s)")
    return ff, ft


# (名稱, key, 時間窗月數 or None=全期, 平滑強度, 退回層)
# "a>b"：用 a 的資料做統計、用 b 去查詢（新建案行情用）
SPEC1 = [
    ("all_24", "k_all", 24, 0, None),
    ("all_6", "k_all", 6, 0, "all_24"),
    ("city", "k_city", None, 50, None),
    ("city_12", "k_city", 12, 50, "city"),
    ("dist", "k_dist", None, 20, "city"),
    ("dist_12", "k_dist", 12, 20, "dist"),
    ("dist_6", "k_dist", 6, 20, "dist_12"),
    ("dtype", "k_dtype", None, 20, "dist"),
    ("dtype_12", "k_dtype", 12, 20, "dist_12"),
    ("dfl", "k_dfl", None, 20, "dist"),
    ("dfl_12", "k_dfl", 12, 20, "dist_12"),
    ("sec", "k_sec", None, 10, "dist_12"),
    ("road", "k_road", None, 10, "dist_12"),
    ("road_12", "k_road", 12, 10, "road"),
    ("seg", "k_seg", None, 10, "road_12"),
    ("road2", "k_road2", None, 10, "dist_12"),
    ("citynew_12", "k_citynew>k_city", 12, 30, "city_12"),
    ("distnew_12", "k_distnew>k_dist", 12, 10, "citynew_12"),
    ("distnew_24", "k_distnew>k_dist", 24, 10, "dist_12"),
    ("dtypenew_12", "k_dtypenew>k_dtype", 12, 10, "distnew_12"),
    ("roadnew_24", "k_roadnew>k_road", 24, 5, "distnew_24"),
```

```
]
SPEC2 = [   # 建案層級，沒資料時退回 te_local（路段 > 地段 > 區近 12 月）
    ("proj", "k_proj", None, 3, "local"),
    ("proj_6", "k_proj", 6, 3, "proj"),
    ("bld", "k_bld", None, 3, "proj"),
]
STALE = {"proj", "road", "dist"}


def te_block(fit, y, q, cutoff):
    """對 q 每列，用 fit 中月份 <= cutoff 的成交計算各層級的平滑平均。"""
    cutoff = np.asarray(cutoff, float)
    gmean = float(y.mean())
    cache = {}
    out = pd.DataFrame(index=q.index)

    def run(spec):
        for name, key, win, m, parent in spec:
            kf, kq = key.split(">") if ">" in key else (key, key)
            if kf not in cache:
                cache[kf] = _cum_stats(fit[kf], fit["mi"], y)
            s, n, last = _lookup(cache[kf], q[kq], cutoff)
            if win:
                s0, n0, _ = _lookup(cache[kf], q[kq], cutoff - win)
                s, n = s - s0, n - n0
            prior = out[f"te_{parent}"].to_numpy() if parent else gmean
            with np.errstate(invalid="ignore", divide="ignore"):
                out[f"te_{name}"] = np.where(n > 0, (s + m * prior) / (n +
m), prior)
            out[f"cnt_{name}"] = n
            if name in STALE:
                out[f"stale_{name}"] = np.where(n > 0, q["mi"].to_numpy() -
last, np.nan)

    run(SPEC1)
    out["te_local"] = np.where(out["cnt_seg"] > 0, out["te_seg"],
                               np.where(out["cnt_sec"] > 0, out["te_sec"],
out["te_dist_12"]))
    run(SPEC2)
    out["gap"] = np.clip(q["mi"].to_numpy() - cutoff, 1, None)          # 資
料延遲月數
    out["te_all_trend"] = out["te_all_6"] - out["te_all_24"]           # 全國
近期漲跌
    out["te_dist_trend"] = out["te_dist_6"] - out["te_dist"]           # 區域
近期漲跌
    out["te_proj_prem"] = out["te_proj"] - out["te_local"]             # 建案
```

```
相對周邊溢價
    out["te_new_prem_dist"] = out["te_distnew_12"] - out["te_dist_12"]  # 新
案相對整區溢價
    out["te_new_prem_road"] = out["te_roadnew_24"] - out["te_road_12"]
    return out


V5_DROP = {"總價元", "area_eff", "mi", "proj_first_mi"}
KEEP_K = {"k_city", "k_dist"}


def v5_build(ff, ft, seed=0):
    """產生特徵 B 的模型矩陣。訓練列的歷史特徵隨機延遲 1~12 個月；測試列用全部歷史。"""
    t0 = time.time()
    ff, y = target(ff)
    rng = np.random.default_rng(seed)
    mi = ff["mi"].to_numpy()
    lag = rng.integers(1, LAG_MAX + 1, size=len(ff))
    enc_fit = te_block(ff, y, ff, mi - lag)
    enc_app = te_block(ff, y, ft, np.full(len(ft), np.nanmax(mi)))
    cats = {c: list(ff[c].astype(str).value_counts().index[:400]) for c in
CAT_COLS}

    def mat(f, enc):
        # 建案、路名等高基數 key 不直接進模型，只透過歷史價格使用
        cols = [c for c in f.columns
                if c not in V5_DROP and (not c.startswith(("_", "k_")) or c
in KEEP_K)]
        X = pd.concat([f[cols].reset_index(drop=True),
enc.reset_index(drop=True)], axis=1)
        for c in CAT_COLS:
            X[c] = pd.Categorical(X[c].astype(str), categories=cats[c])
        return X.loc[:, ~X.columns.duplicated()]

    X_fit, X_app = mat(ff, enc_fit), mat(ft, enc_app)
    print(f"[特徵 B] 歷史特徵(seed={seed}) 完成 ({time.time() - t0:.0f}s), 特徵
數 {X_fit.shape[1]}")
    return X_fit, y, X_app, ff


# ============================================================
# 4. 模型
# ============================================================
def lgb_fit_predict(X_fit, y, X_app, params, n_iter, w=None, cats="auto"):
    m = lgb.LGBMRegressor(n_estimators=n_iter, random_state=0, **params)
    m.fit(X_fit, y, sample_weight=w, categorical_feature=cats)
```

```
    return m.predict(X_app)


def hgb_fit_predict(X_fit, y, X_app, cat_cols):
    """sklearn HGB：類別轉整數編碼；去掉只有一種值或全缺值的欄位（HGB 不接受）。"""
    Zf, Za = X_fit.copy(), X_app.copy()
    for c in cat_cols:
        for Z in (Zf, Za):
            codes = Z[c].cat.codes.astype(float)
            codes[(codes < 0) | (codes >= 250)] = np.nan
            Z[c] = codes
    useful = [c for c in Zf.columns if Zf[c].nunique(dropna=True) > 1]
    Zf, Za = Zf[useful], Za[useful]
    m = HistGradientBoostingRegressor(
        loss="absolute_error", learning_rate=0.1, max_iter=OLD_HGB_MAX_ITER,
        max_leaf_nodes=127, min_samples_leaf=40, l2_regularization=1.0,
        early_stopping=True, validation_fraction=0.1, n_iter_no_change=100,
        categorical_features=[c in cat_cols for c in Zf.columns],
random_state=0)
    m.fit(Zf, y)
    print(f"    HGB 實際使用 {m.n_iter_} 棵樹")
    return m.predict(Za)


def timed(name, fn):
    t0 = time.time()
    out = fn()
    print(f"  [{name}] 完成 ({time.time() - t0:.0f}s)")
    return out


# ============================================================
# 5. 安置住宅規則
# ============================================================
def apply_resettle(test, pred):
    """train 中沒有安置住宅，改用規則：
    總價 = (建物面積 − 車位面積)/3.305785 × PER_PING + 車位數 × PARK − 差額減收金
額"""
    rm = test["備註"].fillna("").astype(str)
    mask = rm.str.contains("安置").to_numpy()
    a_b = num(test, "建物移轉總面積平方公尺").fillna(0)
    a_p = num(test, "車位移轉總面積平方公尺").fillna(0)
    n_p = test["交易筆棟數"].astype(str).str.extract(r"車位(\d+)")
[0].astype(float).fillna(0)
    d = rm.str.extract(r"原房屋價款(\d+)元.*?減收後之房屋價款為(\d+)
元").astype(float)
```

```
    disc = (d[0] - d[1]).fillna(0)
    rule = ((a_b - a_p).clip(lower=0) / PING * PER_PING + n_p * PARK -
disc).to_numpy()
    out = pred.copy()
    out[mask] = rule[mask]
    print(f"安置住宅規則套用 {mask.sum():,} 筆")
    return out


# ============================================================
# 6. 主程式
# ============================================================
def main():
    t_all = time.time()
    full, test = load()
    logp = {}   # 每個模型預測的 log 單價

    # ---------- 特徵 A：4 個模型 ----------
    of, ot = old_prepare(full, test)
    X, y, Xt = old_build(of, ot, seed=42)
    logp["old_lgb"] = timed("old_lgb", lambda: lgb_fit_predict(
        X, y, Xt, OLD_LGB_PARAMS, OLD_LGB_ITER, cats=O_CAT))
    logp["old_l2"] = timed("old_l2", lambda: lgb_fit_predict(
        X, y, Xt, dict(OLD_LGB_PARAMS, objective="regression"), 1500,
cats=O_CAT))
    logp["old_hgb"] = timed("old_hgb", lambda: hgb_fit_predict(X, y, Xt,
O_CAT))
    del X, Xt

    X, y, Xt = old_build(of, ot, seed=7)   # 換一組隨機延遲 → 不同的歷史特徵
    logp["old_lgb_s7"] = timed("old_lgb_s7", lambda: lgb_fit_predict(
        X, y, Xt, OLD_LGB_PARAMS, OLD_LGB_ITER, cats=O_CAT))
    del X, Xt, of, ot

    # ---------- 特徵 B：3 個模型 ----------
    ff, ft = v5_prepare(full, test)
    X, y, Xt, fk = v5_build(ff, ft, seed=0)
    for v in V5_VARIANTS:
        Xf, Xa = X, Xt
        if not v["kdist_cat"]:
            Xf, Xa = X.drop(columns=["k_dist"]), Xt.drop(columns=["k_dist"])
        w = None
        if v["weight"]:   # 依總價加權，讓高價物件在訓練中更受重視
            tot = fk["總價元"].to_numpy()
            w = np.clip(tot / np.median(tot), 0.1, 10)
        params = dict(V5_PARAMS, objective=v["objective"])
```

```
        logp[v["name"]] = timed(v["name"], lambda: lgb_fit_predict(
            Xf, y, Xa, params, v["n_iter"], w=w))
    del X, Xt

    # ---------- 加權幾何平均（對 log 單價做加權平均，再乘回面積） ----------
    tot_w = sum(WEIGHTS.values())
    mean_log = sum(w * logp[name] for name, w in WEIGHTS.items()) / tot_w
    pred = np.exp(mean_log) * area_eff(test)

    # ---------- 安置住宅規則 ----------
    pred = apply_resettle(test, pred)

    sub = pd.DataFrame({"編號": test["編號"].values, "總價元": np.round(pred,
2)})
    assert len(sub) == len(test) and sub["總價元"].notna().all()
    sub.to_csv(OUT_FILE, index=False, encoding="utf-8-sig")
    print(f"\n寫出 {OUT_FILE.name} ({len(sub):,} 筆)，共 {time.time() -
t_all:.0f}s")


if __name__ == "__main__":
    main()
```